Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date:**	**Enrolment No:92410133020**

Aim: To study N-gram model

IDE: Google Colab

Theory:

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities. Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff. An N-gram model is one type of a Language Model (LM), which is about finding the probability distribution over word sequences.

Consider two sentences: "There was heavy rain" vs. "There was heavy flood". From experience, we know that the former sentence sounds better. An N-gram model will tell us that "heavy rain" occurs much more often than "heavy flood" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called unigram. If a model considers only the previous word to predict the current word, then it's called bigram. If two previous words are considered, then it's a trigram model.

An n-gram model for the above example would calculate the following probability:

P('There was heavy rain') = P('There', 'was', 'heavy', 'rain') = P('There')P('was'|'There')P('heavy'|'There was')P('rain'|'There was heavy')

Since it's impractical to calculate these conditional probabilities, using Markov assumption, we approximate this to a bigram model: P('There was heavy rain') ~ P('There')P('was'|'There')P('heavy'|'was')P('rain'|'heavy') In speech recognition, input may be noisy and this can lead to wrong speech-to-text conversions.

N-gram models can correct this based on their knowledge of the probabilities. Likewise, N-gram models are used in machine translation to produce more natural sentences in the target language.

When correcting for spelling errors, sometimes dictionary lookups will not help. For example, in the phrase "in about fifteen mineuts" the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram models can correct such errors.

N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix. By looking at N-gram statistics, we could also classify languages or differentiate

between US and UK spellings. For example, 'sz' is common in Czech; 'gb' and 'kp' are common in Igbo. In general, many NLP applications benefit from N-gram models including part-of-speech tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Program (Code):

To be attached with

1. N-Gram Model Code for generating dynamic N-Grams

```
# Import libraries
import nltk
from nltk.util import ngrams
from collections import Counter, defaultdict
import re

nltk.download('punkt')
nltk.download('punkt_tab')

# Sample text corpus
text = """There was heavy rain. There was heavy wind.
There was heavy rain and storm."""

# Preprocessing
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    tokens = nltk.word_tokenize(text)
    return tokens

tokens = preprocess(text)

# Function to generate N-grams
def generate_ngrams(tokens, n):
    return list(ngrams(tokens, n))

# Bigram model
n = 2
bigrams = generate_ngrams(tokens, n)

# Count frequencies
```

```
bigram_counts = Counter(bigrams)
unigram_counts = Counter(tokens)

# Probability calculation
def calculate_bigram_prob(w1, w2):
    return bigram_counts[(w1, w2)] / unigram_counts[w1]

# Display bigrams
print("Bigrams and Counts:")
for bg, count in bigram_counts.items():
    print(bg, ":", count)

# Predict next word
def predict_next_word(word):
    candidates = {}
    for (w1, w2) in bigram_counts:
        if w1 == word:
            candidates[w2] = calculate_bigram_prob(w1, w2)
    return sorted(candidates.items(), key=lambda x: x[1], reverse=True)

# Example prediction
word = "heavy"
prediction = predict_next_word(word)

print("\nNext word predictions after 'heavy':")
for w, prob in prediction:
    print(f"{w} : {prob:.2f}")
```

Results:

To be attached with

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Bigrams and Counts:
('there', 'was') : 3
('was', 'heavy') : 3
('heavy', 'rain') : 2
('rain', 'there') : 1
('heavy', 'wind') : 1
('wind', 'there') : 1
('rain', 'and') : 1
('and', 'storm') : 1

Next word predictions after 'heavy':
rain : 0.67
wind : 0.33
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study N-gram model	
Experiment No: 7	**Date:**	**Enrolment No:92410133020**

Observation:

- The model correctly predicts "rain" as the most probable word after "heavy".
- This is because "heavy rain" occurs more frequently than "heavy wind".
- N-gram models depend heavily on training data frequency.
- They are simple but effective for:
  - Text prediction
  - Speech recognition
  - Spell correction

Limitations:

- Cannot capture long-range dependencies
- Suffers from data sparsity (fixed using smoothing)

Post Lab Exercise:

Apply the probability distribution concept to predict the next word by taking sample document and seed phrases. Attach the code and screenshot. Write your observation here.

```
def predict_sentence(seed, n=2):
    words = preprocess(seed)
    last_word = words[-1]

    predictions = predict_next_word(last_word)

    if predictions:
        print(f"\nPredicted next words for '{seed}':")
        for word, prob in predictions:
            print(f"{word} : {prob:.2f}")
    else:
        print("No prediction available.")

# Test
predict_sentence("there was heavy")
```

```
# Test
predict_sentence("there was heavy")
```

```
...
Predicted next words for 'there was heavy':
rain : 0.67
wind : 0.33
```